4. Types of Operating System & Services (Expanded)

More on OS Types:

Let's add a couple more types that often pop up in exams:

- **Network OS:** Runs on a server and manages data, users, groups, and security over a local network. (Like a manager overseeing an entire office building). *Examples: Windows Server, Linux*.
- **Mobile OS:** Specifically designed for smartphones and tablets, focusing on touch interfaces, battery management, and cellular connectivity. *Examples: Android, iOS*.

More on OS Services:

Beyond just running apps, the OS does a lot of background management:

- **Resource Allocation:** When multiple users or jobs are running, the OS acts like a referee, deciding who gets to use the CPU, memory, and printers so everyone gets a fair turn.
- **Protection and Security:** * *Protection:* Ensuring that one app doesn't accidentally overwrite the memory of another app.
  - *Security:* Keeping unauthorized users out through passwords and permissions.

5. System Calls & System Boot (Expanded)

The "How" of System Calls: User Mode vs. Kernel Mode

To understand system calls, you need to understand the two "modes" a computer runs in:

1. **User Mode:** Where your normal applications (like a web browser or a game) run. It has restricted privileges.
2. **Kernel Mode:** Where the core OS runs. It has total access to all hardware and memory.

When an app in User Mode needs to access hardware (like saving a file), it triggers a **System Call**. This causes a **Context Switch**: the CPU temporarily pauses User Mode, switches into Kernel Mode, does the sensitive task, and then switches back to User Mode to give the app its results.

(Think of it like a regular employee (User Mode) who needs a document from a locked safe. They must ask the manager (Kernel Mode) to open the safe for them.)

Expanding on System Boot (The Boot Sector):

When the BIOS looks for the bootloader, it specifically looks at the very first sector of your hard drive, often called the **MBR (Master Boot Record)** or the newer **EFI System Partition**. If this tiny sector gets corrupted, your computer will turn on, but it will say "Operating System Not Found."

6. Process States & Scheduling (Expanded)

How the OS tracks states: The Process Control Block (PCB)

How does the OS remember exactly what an app was doing when it paused it to let another app run? It uses a **PCB**.

Think of the PCB as a highly detailed ID card or "medical chart" for every single process. It contains:

- **Process State:** (Is it ready, running, or waiting?)
- **Program Counter:** Exactly which line of code it needs to execute next.
- **CPU Registers:** Temporary data it was holding.
- **Memory limits:** How much RAM it is allowed to use.

Process Scheduling Algorithms (How the CPU chooses who goes next):

The Short-Term Scheduler uses specific rules (algorithms) to pick the next process. Here are the most famous ones:

1. **FCFS (First-Come, First-Served):** The fairest but often slowest. Whoever asks for the CPU first gets it until they are done. (Like a line at a grocery store).
2. **SJF (Shortest Job First):** The CPU looks at the queue and picks the task that will take the least amount of time. Great for speed, but long tasks might wait forever (this is called *starvation*).
3. **Round Robin (RR):** The most common for modern computers. Every process gets a tiny, equal slice of time (e.g., 10 milliseconds). If it's not done in that time, it gets paused and sent to the back of the line, and the next process goes. (Like a teacher giving every student 1 minute to speak before rotating).